

Organización Detallada del Trabajo

Práctica Final conjunta: Metodologías de la Programación + Estructuras de Datos

Para qué sirve: este documento explica de forma práctica qué hay que construir, cómo se va a trabajar, qué tecnología se usará, qué hará cada persona, qué archivos tocará y cómo se integrará todo sin pisarse.

Documentos de referencia: enunciado original, `prd.md`, `plan.md`, `AGENTS.md`, `skills.md` y `reparto-trabajo.md`.

Regla principal: no se empieza a programar una parte si antes no están claras sus decisiones, su responsable, sus archivos y sus pruebas.

1. En qué consiste el proyecto

El proyecto es un juego de exploración por turnos en Java con interfaz gráfica JavaFX. El jugador empieza en una habitación inicial y debe llegar a una puerta de salida exterior antes de quedarse sin vida o sin turnos.

El mapa global no es una lista de pantallas: es un **grafo propio**. Cada nodo del grafo es una habitación. Las aristas indican conexiones entre habitaciones mediante puertas, escaleras, portales u otros elementos equivalentes.

Cada habitación no es un nodo abstracto: contiene una **matriz bidimensional** de celdas. Las habitaciones pueden tener tamaños distintos. Cada celda puede estar vacía o contener un elemento del juego.

Elementos mínimos del juego

Elemento	Qué debe representar	Regla importante
Jugador	Vida, ataque, defensa, movimiento, inventario, habitación actual y coordenadas.	Solo puede hacer como máximo un movimiento y una acción por turno.
Habitación	Matriz de celdas y elementos colocados.	Debe representarse completa en JavaFX.
Celda	Contenido de una posición: vacío, enemigo, objeto, puerta, trampa o salida.	Una celda no debe contener múltiples entidades principales a la vez.
Enemigo	Vida, ataque, defensa, movimiento y posición.	Actúa después del jugador y se mueve hacia él dentro de la habitación.
Objeto	Nombre, tipo, modificadores, si es equipable o fungible.	Puede modificar ataque, defensa, vida o movimiento.
Puerta o salida	Conexión con otra habitación o salida final.	Al abrir una puerta se cambia de habitación y termina el turno.

Reglas de juego que no se deben olvidar

- El movimiento diagonal directo está prohibido.
- El jugador solo puede escoger una casilla destino entre las alcanzables en ese turno.
- Las acciones de atacar, recoger objeto o abrir puerta se hacen sobre celdas adyacentes cuando corresponda.
- La defensa es automática ante ataques enemigos.
- La fórmula de daño es: $\text{vida} = \text{vida} - \max(0, \text{ataque} * (\text{aleatorio} * 2) - \text{defensa})$.
- La vida nunca debe quedar por debajo de cero.
- Se debe informar constantemente de la distancia mínima hacia la puerta adecuada y del número mínimo de habitaciones hasta la salida.
- El jugador puede comprar ver el camino en pantalla, pero hay que decidir antes qué significa comprarlo.

2. Qué se va a usar

Parte	Herramienta o decisión	Notas
Lenguaje	Java	Todo el código principal será Java.
Interfaz	JavaFX	La matriz puede representarse con <code>GridPane</code> . Claridad antes que estética.
Pruebas	JUnit	Obligatorio para estructuras, lógica, modelo y persistencia. No hace falta probar visualmente JavaFX con JUnit.
Persistencia	JSON	Hay que decidir librería permitida antes de programar.
Arquitectura	Separación por paquetes	<code>model</code> , <code>structures</code> , <code>logic</code> , <code>persistence</code> , <code>ui</code> , <code>exceptions</code> , <code>log</code> , <code>app</code> .
Mapa	Grafo propio	No usar estructuras estándar equivalentes para resolver esta parte.
Habitaciones	Matriz propia	Hay que decidir si se implementa con nodos, arrays internos permitidos o envoltorio propio.
Turnos	Cola propia	Jugador primero, después enemigos de la habitación.
Inventario	Lista propia u otra estructura propia	No usar <code>ArrayList</code> , <code>LinkedList</code> ni equivalentes.
Acciones	Árbol propio recomendado	Uso propuesto: organizar posibles acciones del turno.

Restricción crítica: no usar `ArrayList`, `HashMap`, `LinkedList` ni estructuras estándar equivalentes en las partes evaluadas de Estructuras de Datos. Usarlas para ahorrar tiempo puede suponer un 0 en esa parte.

3. Cómo se va a trabajar

1. Primero se cierran dudas y decisiones: grafo dirigido/no dirigido, JSON, librería, matriz propia, árbol, recurso para comprar camino y mínimos de contenido.
2. Después se redacta la especificación: requisitos, casos de uso, invariantes, contratos, modelo de dominio y bocetos.
3. Después se prepara UML obligatorio antes de programar en serio.
4. Se crea el proyecto Java base con JavaFX y JUnit.
5. Se implementan estructuras propias y se prueban aisladas.
6. Se implementa el modelo y la lógica sin JavaFX.
7. Se implementa JSON cuando el modelo esté estable.
8. Se implementa JavaFX al final, usando la lógica ya probada.
9. Se integra por bloques, ejecutando tests en cada bloque.

10. Se prepara memoria, diario de IA, ZIP, GitHub y vídeo.

4. Estructura de carpetas y responsabilidad

Ruta	Qué contiene	Responsable principal
<code>pom.xml</code> o <code>build.gradle</code>	Configuración de proyecto, JavaFX y JUnit.	Persona 2
<code>src/main/java/.../app</code>	Arranque de la aplicación.	Persona 3, con base de Persona 2 si procede
<code>src/main/java/.../model</code>	Jugador, enemigo, objeto, celda, habitación, partida.	Persona 1 y Persona 3 coordinadas
<code>src/main/java/.../structures</code>	Lista, pila, cola, lista circular, árbol, grafo, matriz.	Persona 2
<code>src/main/java/.../logic</code>	Turnos, movimiento, combate, rutas, victoria, derrota.	Persona 3 para reglas; Persona 2 para algoritmos de rutas/BFS
<code>src/main/java/.../persistence</code>	Lectura y escritura JSON.	Persona 3
<code>src/main/java/.../ui</code>	Vistas JavaFX y controladores.	Persona 3
<code>src/main/java/.../exceptions</code>	Excepciones personalizadas.	Persona 1 y Persona 3
<code>src/main/java/.../log</code>	Registro de eventos del juego.	Persona 3
<code>src/main/resources</code>	JSON de ejemplo y recursos.	Persona 3
<code>src/test/java/.../structures</code>	Tests de estructuras.	Persona 2
<code>src/test/java/.../model</code>	Tests de invariantes y dominio.	Persona 1
<code>src/test/java/.../logic</code>	Tests de turnos, movimiento, combate y rutas.	Persona 2 y Persona 3
<code>src/test/java/.../persistence</code>	Tests de carga y guardado JSON.	Persona 3
<code>docs</code>	Memoria, UML, bocetos y diario de IA.	Persona 1, con aportes de todos

5. Persona 1: metodología, modelo base y documentación

Objetivo de su trabajo

Garantizar que el proyecto está bien definido, que el diseño se puede explicar y que la memoria refleja lo que realmente se ha implementado.

Tareas concretas

Orden	Tarea	Resultado que debe entregar
1	Revisar dudas del PRD y preparar decisiones para el grupo.	Lista de decisiones: grafo, JSON, árbol, matriz, objetos mínimos, enemigos y compra del camino.
2	Redactar requisitos definitivos.	Requisitos funcionales y no funcionales separados de opcionales.
3	Crear casos de uso.	Casos para mover, cambiar habitación, atacar, recoger, usar objeto, guardar, cargar y terminar partida.
4	Definir invariantes.	Jugador en celda válida, vida no negativa, celda sin múltiples entidades, objeto no duplicado.
5	Definir contratos o interfaces principales.	Interfaces de estructuras, servicios de lógica y persistencia si se deciden.
6	Crear UML obligatorio.	Casos de uso, clases, secuencia, estados y actividad.
7	Diseñar bocetos de interfaz.	Pantalla principal con matriz, estado, inventario, acciones y log.
8	Definir modelo base con Persona 3.	Lista de clases, atributos, responsabilidades e invariantes.
9	Crear tests del modelo.	Tests de vida, posición, celda, inventario básico y validaciones.
10	Mantener memoria y diario de IA.	Memoria coherente, crítica del proyecto y entradas de IA completas.

Archivos que toca

- `docs/memoria.md` o documento de memoria.
- `docs/diario-ia.md`.
- `docs/uml` y `docs/bocetos`.
- `src/main/java/.../model`, solo clases acordadas.
- `src/main/java/.../exceptions`, para excepciones del dominio.
- `src/test/java/.../model`.

Qué debe pedir a Opencode

- “Usa los documentos del proyecto. Aplica la skill de especificar requisitos y casos de uso. No programes nada.”
- “Revisa estos casos de uso contra el PRD y dime qué falta.”
- “Crea un borrador de diagrama de clases coherente con estas clases, sin inventar funcionalidades.”
- “Crea tests JUnit para invariantes del modelo: vida no negativa, posición válida y celda sin múltiples entidades.”

No debe tocar sin coordinar

- No modificar estructuras propias.
- No cambiar reglas de movimiento, combate o turnos ya implementadas.
- No cambiar formato JSON sin Persona 3.
- No cambiar JavaFX salvo bocetos o documentación.

6. Persona 2: proyecto base, estructuras y algoritmos

Objetivo de su trabajo

Garantizar que la parte de Estructuras de Datos está implementada con estructuras propias, probada, justificada y usada de verdad en el juego.

Tareas concretas

Orden	Tarea	Resultado que debe entregar
1	Crear proyecto base si se le asigna.	Maven o Gradle, JavaFX, JUnit, carpetas y comandos reales.
2	Implementar lista enlazada propia.	Operaciones mínimas: añadir, eliminar, buscar, obtener, tamaño y vacía.
3	Implementar pila propia.	Apilar, desapilar, cima, vacía y tamaño.
4	Implementar cola propia.	Encolar, desencolar, frente, vacía y tamaño. Uso previsto: turnos.
5	Implementar lista circular propia.	Uso previsto: rotación de elementos, turnos secundarios o recorrido cíclico si se decide.
6	Implementar árbol propio.	Uso recomendado: organizar acciones posibles del jugador.
7	Implementar grafo propio.	Nodos de habitación, conexiones, vecinos y búsqueda.
8	Implementar matriz propia.	Acceso por fila/columna, validación de coordenadas y contenido de celdas.
9	Implementar BFS.	Casillas alcanzables dentro de una habitación sin diagonal directo.
10	Implementar ruta mínima entre habitaciones.	Número mínimo de habitaciones hasta la salida y puerta adecuada.
11	Crear tests JUnit.	Tests de cada estructura, BFS, rutas y casos borde.
12	Documentar costes.	Tabla con coste de operaciones principales y justificación de uso.

Archivos que toca

- `pom.xml` o `build.gradle`, solo configuración aprobada.
- `src/main/java/.../structures`.
- `src/main/java/.../logic`, solo algoritmos de rutas, BFS y cálculo de alcanzables.
- `src/test/java/.../structures`.
- `src/test/java/.../logic`, solo tests de algoritmos.
- Sección de estructuras y costes en memoria.

Qué debe pedir a Opendcode

- “Aplica la skill crear proyecto base. Configura JavaFX y JUnit. No implementes reglas del juego.”
- “Aplica la skill implementar una estructura de datos para la cola propia. Crea tests JUnit.”
- “Revisa `structures` y dime si hay estructuras prohibidas o costes mal documentados.”
- “Implementa solo BFS para casillas alcanzables usando estructuras propias. No cambies combate ni UI.”

No debe tocar sin coordinar

- No modificar JavaFX ni controladores.
- No cambiar reglas de combate, objetos o inventario.
- No cambiar modelo de dominio sin Persona 1.
- No cambiar formato JSON sin Persona 3.
- No introducir dependencias externas innecesarias.

7. Persona 3: lógica, JSON e interfaz

Objetivo de su trabajo

Convertir el modelo y las estructuras en un juego funcional, persistente y usable desde JavaFX sin mezclar la lógica con la interfaz.

Tareas concretas

Orden	Tarea	Resultado que debe entregar
1	Integrar modelo base con Persona 1.	Jugador, enemigo, objeto, celda, habitación y partida listos para lógica.
2	Implementar gestor de turnos.	Jugador primero, después enemigos; máximo un movimiento y una acción.
3	Integrar movimiento.	Usa BFS y rutas de Persona 2; no reimplementa estructuras.
4	Implementar acciones.	Recoger, usar, equipar, atacar, abrir puerta y terminar turno.
5	Implementar combate.	Fórmula de daño, defensa automática, modificadores y vida no negativa.
6	Implementar enemigos.	Movimiento básico hacia el jugador y ataque si procede.
7	Implementar victoria y derrota.	Victoria por salida; derrota por vida o turnos.
8	Implementar log.	Registro de movimientos, objetos, ataques, puertas, errores y final de partida.
9	Definir e implementar JSON.	Configuración inicial, guardado y carga de partida.
10	Crear JSON de ejemplo.	Archivo de configuración y, si procede, partida de ejemplo.
11	Crear interfaz JavaFX.	Matriz, estado, inventario, acciones, log y camino mínimo.
12	Crear tests.	Tests de lógica, combate, turnos, JSON y errores.

Archivos que toca

- `src/main/java/.../model`, coordinado con Persona 1.
- `src/main/java/.../logic`.
- `src/main/java/.../persistence`.
- `src/main/java/.../ui`.
- `src/main/java/.../app`.
- `src/main/java/.../log`.
- `src/main/resources/config-ejemplo.json`.
- `src/main/resources/partida-ejemplo.json`.
- `src/test/java/.../logic` y `src/test/java/.../persistence`.

Qué debe pedir a Opencode

- “Implementa solo la lógica de turnos usando la cola propia ya existente. No modifiques estructuras.”
- “Crea tests para movimiento: destino válido, fuera de rango, diagonal prohibida y celda ocupada.”
- “Aplica la skill implementar persistencia JSON según este formato aprobado. No cambies el modelo sin avisar.”
- “Crea una vista JavaFX sencilla con matriz, estado, inventario, acciones y log. Mantén la lógica fuera de la UI.”

No debe tocar sin coordinar

- No modificar estructuras propias salvo bug acordado con Persona 2.
- No cambiar requisitos, UML o memoria sin Persona 1.
- No cambiar formato JSON después de tener tests sin avisar.
- No meter reglas de juego directamente dentro de controladores JavaFX.

8. Decisiones que hay que cerrar antes de programar

Decisión	Por qué importa	Propuesta mínima
Grafo dirigido o no dirigido	Afecta a rutas, puertas y JSON.	No dirigido salvo que una puerta sea explícitamente de sentido único.
Costes de conexiones	Afecta a BFS o Dijkstra.	Coste fijo para evitar Dijkstra salvo que el profesor lo exija.
Matriz propia	Afecta a Estructuras de Datos.	Clase propia con operaciones claras y costes documentados.
Arrays nativos	Puede afectar a restricciones.	Confirmarlo con profesor antes de usarlos internamente.
Librería JSON	Afecta a dependencias.	Usar una librería sencilla permitida, o JSON manual si no se permite.
Compra de camino	Requisito funcional ambiguo.	Consumir un objeto o un turno, pero decidirlo y documentarlo.
Uso del árbol	Debe justificarse.	Árbol de acciones posibles del jugador.
Contenido mínimo	Evita quedarse corto.	Definir mínimo de habitaciones, enemigos, objetos, puertas y trampas.

9. Formato JSON mínimo

Configuración inicial

- Habitaciones: identificador, filas, columnas y contenido de celdas.
- Grafo: conexiones entre habitaciones y puertas asociadas.
- Jugador: habitación inicial, coordenadas, vida, ataque, defensa y movimiento.
- Enemigos: habitación, coordenadas, vida, ataque, defensa y movimiento.
- Objetos: identificador, tipo, modificadores, equipable/fungible y posición inicial.
- Objetivo: habitaciones o puertas de salida.
- Turnos: contador máximo de partida.

Estado de partida

- Estado actual del jugador, inventario y objetos equipados.
- Habitación actual y coordenadas.
- Estado de enemigos vivos o derrotados.
- Objetos recogidos o eliminados del mapa.
- Estado de puertas, trampas y elementos interactivos.
- Turnos restantes o consumidos.
- Estado de partida: en curso, victoria o derrota.

10. Orden de integración

1. Se aprueban decisiones abiertas y se apuntan para la memoria.
2. Persona 2 crea proyecto base y comandos reales.
3. Persona 2 integra estructuras propias y tests.
4. Persona 1 integra modelo base e invariantes.
5. Persona 2 integra BFS y rutas.
6. Persona 3 integra turnos y movimiento usando lo anterior.
7. Persona 3 integra combate, objetos, inventario, victoria y derrota.
8. Persona 3 integra JSON cuando el modelo esté estable.
9. Persona 3 integra JavaFX cuando la lógica pase tests.
10. Los tres ejecutan pruebas finales y revisan memoria, UML y diario.

11. Protocolo para no pisarse

- Cada persona trabaja principalmente en sus paquetes asignados.
- Antes de cambiar una API compartida, se avisa al grupo.
- Antes de tocar `model`, Persona 1 y Persona 3 acuerdan cambios.
- Antes de tocar `logic` de movimiento, Persona 2 y Persona 3 acuerdan límites.
- Antes de tocar JSON, Persona 3 avisa si cambia el formato.
- Antes de integrar, se ejecutan los tests relacionados.
- Si Opencode genera código, la persona responsable debe revisarlo y entenderlo.
- Todo uso de IA se registra en el diario.

12. Checklist de integración por bloque

Bloque	Para considerarlo terminado
Proyecto base	Compila, ejecuta tests y los comandos están documentados en <code>AGENTS.md</code> .
Estructuras	Tienen tests, costes documentados y no usan estructuras prohibidas.
Modelo	Invariantes probados y clases coherentes con UML.
Lógica	Turnos, movimiento, combate, victoria y derrota tienen tests.
JSON	Carga configuración, guarda partida, carga partida y controla errores.
JavaFX	Muestra matriz, jugador, inventario, acciones y log sin mezclar reglas.
Documentación	Memoria, UML, bocetos, diario de IA y crítica están actualizados.

13. Checklist final común

Área	Comprobación
Ejecución	El proyecto compila, JavaFX arranca y se puede jugar una partida completa.
Victoria/derrota	Se puede ganar por salida y perder por vida o por turnos.
Movimiento	Se iluminan casillas alcanzables y no hay diagonal directo.
Turnos	Máximo un movimiento y una acción; jugador primero, enemigos después.
Inventario	Se ve constantemente y permite usar/equipar objetos.
Combate	Usa fórmula correcta, defensa automática y vida no negativa.
Rutas	Informa distancia mínima y habitaciones mínimas hasta la salida.
Persistencia	JSON de configuración, guardar partida y cargar partida funcionan.
Estructuras	Lista, pila, cola, lista circular, árbol, grafo y matriz existen, se usan y tienen tests.
Restricciones	No hay <code>ArrayList</code> , <code>HashMap</code> , <code>LinkedList</code> ni equivalentes prohibidos en partes evaluadas.
Memoria	Incluye portada con todos, crítica del proyecto, decisiones, UML, bocetos, pruebas y costes.
IA	Diario completo con agentes, skills, prompts, resultados, modificaciones, crítica y reajustes.
Vídeo	Aparecen todos, se muestran funcionalidades y se puede subir a Blackboard en menos de 100 MB o dividido.
Entrega	ZIP, GitHub, JSON de ejemplo y entregas separadas para ambas asignaturas.

14. Reparto del vídeo

- Persona 1 explica requisitos, metodología, UML, bocetos, memoria y uso de IA.
- Persona 2 explica estructuras propias, costes, restricciones, BFS, rutas y tests de estructuras.
- Persona 3 explica partida, reglas, JSON, JavaFX y demostración funcional.
- Los tres deben aparecer y poder explicar el flujo completo.

15. Riesgos principales

- Usar estructuras prohibidas por comodidad.
- Empezar JavaFX antes de tener lógica probada.
- No justificar bien el árbol propio.
- Cambiar el modelo y romper JSON, UML o UI.

- No registrar bien el uso de IA.
- Que el vídeo no cumpla las condiciones de Blackboard.
- Que una persona no pueda explicar su parte.

16. Dudas que conviene confirmar con el profesor

- Fecha real de entrega, porque el PDF dice 2026 pero marca 28/5/2025.
- Si se permiten arrays nativos dentro de estructuras propias.
- Qué librería JSON se puede usar.
- Si el árbol debe tener un uso funcional fuerte o basta con organizar acciones posibles.
- Si hay número mínimo de habitaciones, objetos, enemigos o puertas.
- Si el grafo puede ser no dirigido y sin pesos.

Forma recomendada de avanzar: trabajar en tareas pequeñas, pedir a OpenCode una sola cosa cada vez, ejecutar pruebas relacionadas y registrar el uso de IA en el diario. No integrar cambios grandes sin revisión del grupo.

Este documento es una guía de organización. Si hay contradicción, prevalecen el enunciado original y los documentos técnicos del proyecto: `prd.md`, `plan.md`, `AGENTS.md` y `skills.md`.